

Project Interim Report
On
Global Currency Converter
(Using Core Java & OOP Concepts)

RU-100-01-00012
Object Oriented Programming with Java

SESSION: 2025-26



Submitted By
Priyanshu Dewangan
ERP : RU-25-11044

**Bachelor of Technology in Computer Science &
Engineering with AI and ML in association with Google**

Under the Guidance of
Ashish Shivhare

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI, CG

(April 2026)

Abstract

The **Global Currency Converter** is a Java-based application developed using the principles of **Object-Oriented Programming (OOP)**, specifically focusing on the utility class design pattern. In an increasingly interconnected global economy, real-time financial tools are essential for travelers, investors, and e-commerce platforms to navigate fluctuating exchange rates accurately. This project addresses the need for a lightweight, efficient, and centralized system to perform currency conversions without the overhead of complex state management.

The system utilizes **static members** to store exchange rate constants and provides conversion logic through static methods, allowing the program to function without the need for object instantiation. By encapsulating conversion rates (such as USD, EUR, and GBP to INR) as `static final` variables, the application ensures data integrity and high performance through optimized memory usage. Users can input a specific amount and target currency, and the system instantly computes the equivalent value in Indian Rupees (INR). Ultimately, this project demonstrates how static architecture can be used to create reusable, utility-style software that models real-world financial operations with precision and modularity.

Introduction

Same information about your project but in details you can also add some of the features of your project Objectives (what your project does)

- Manage book records
- Track issued books
- Reduce manual work
- Apply OOP concepts

The **Global Currency Converter** is a specialized financial utility application designed to simplify the complexities of international currency exchange through a robust, programmatic approach. In the modern era of digital finance, the ability to quickly and accurately calculate currency values is fundamental for international trade, travel, and personal finance management.

This project is built using the **Utility Class Pattern** in Java, which prioritizes performance and centralized logic. Unlike traditional applications that create multiple objects to handle tasks, this system utilizes **static members** to maintain a "single source of truth" for exchange rates. This means the application does not need to use memory to create a new "converter" every time a calculation is needed; instead, it provides a direct, high-speed path to the conversion logic. By encapsulating financial data within a secure, controlled class structure, the project ensures that exchange rates remain constant and protected throughout the program's execution.

Key Features

- **Zero-Instance Architecture:** Utilizes static methods and variables to perform calculations without the overhead of object creation, optimizing CPU and RAM usage.
 - **Centralized Rate Management:** All exchange rates are defined as `static final` constants, making it easy to maintain and update rates in one location.
 - **Case-Insensitive Input Handling:** The system is designed to accept currency codes in any format (e.g., "usd", "USD", "Usd"), enhancing user experience and reducing input errors.
 - **Error Guarding:** Built-in validation logic identifies unsupported currency types and provides clear feedback to the user rather than crashing the program.
 - **Standardized Output:** Formats financial results into a clear, readable INR (Indian Rupee) format for consistency.
-

Objectives

The primary goals of the Global Currency Converter project are:

- **Automate Financial Calculations:** To replace manual multiplication and look-up tables with an automated system that provides instant results.
- **Apply OOP Concepts:** To practically demonstrate the use of **Static Variables**, **Static Methods**, **Encapsulation**, and **Utility Classes** in a real-world scenario.
- **Reduce Manual Work:** To eliminate human error associated with calculating multi-currency conversions manually.
- **Improve Code Modularity:** To create a reusable "engine" that can be easily integrated into larger banking or e-commerce software projects.
- **Efficient Resource Management:** To showcase how choosing the right memory storage (Stack vs. Heap vs. Method Area) can make a program more efficient.

Scope

This project is suitable for small libraries and can be extended with database integration in future.

The scope of this project defines the boundaries, features, and limitations of the **Global Currency Converter**. It outlines what the system is designed to handle and how it can be scaled for future use.

1. Functional Scope

The functional scope details the specific features and operations the system performs:

- **Static Currency Logic:** Implementation of a "Utility Class" that provides conversion services without requiring the overhead of creating new objects.
- **Direct Multi-Currency Support:** Hardcoded support for primary global currencies, specifically **USD, EUR, and GBP**, targeted toward conversion into **INR**.
- **Input Validation:** The system is scoped to handle basic error checking, such as identifying unsupported currency codes and returning a null or zero value to prevent system crashes.
- **Mathematical Precision:** Performing basic arithmetic operations using `double` data types to ensure decimal accuracy for financial calculations.

2. Technical Scope

This project serves as a practical application of specific Java/OOP concepts:

- **Memory Management:** Utilizing the **Static Memory Area** (Method Area) for constants, ensuring that exchange rates are stored only once for the entire application lifecycle.
- **Access Control:** Implementing **Encapsulation** through the use of `public static final` modifiers to create read-only constants that cannot be altered during runtime.
- **Procedural-to-OOP Bridge:** Demonstrating how to use a Class as a namespace for related functions (methods) rather than just a blueprint for objects.

3. Out-of-Scope (Limitations)

To keep the project focused on OOP fundamentals, the following are currently excluded:

- **Real-time API Integration:** The system uses hardcoded constants rather than fetching live rates from a financial API (e.g., Fixer.io or XE.com).
- **Database Connectivity:** No persistent storage is used; once the program terminates, transaction history is not saved.

- **GUI Development:** The current scope is limited to a **Command Line Interface (CLI)** for input and output.
- **Reverse Conversion:** The logic is primarily designed for Currency-to-INR; INR-to-Foreign currency is not part of the initial version.

4. Future Enhancements (Scalability)

While the current scope is focused on static members, the architecture allows for:

- **Expansion of Currency Set:** Adding more constants (JPY, CAD, AUD) without altering the core logic.
- **Dynamic Updates:** Introducing a method to update the `static` rates during runtime via an admin-level function.
- **Batch Processing:** Allowing the system to convert an array or list of multiple amounts in a single execution.

System Requirements

Hardware: Computer with 4GB RAM

Software: Java JDK, IDE (VS Code/Eclipse/Edit Plus)

Methodology

Explain how your project works like first we add something, then search, then modify etc.

1. System Initialization (Loading Constants)

Before the user even interacts with the program, the Java Virtual Machine (JVM) loads the `ConverterEngine` class.

- **Action:** The `static final` exchange rates are initialized in the **Method Area** of the memory.
- **Result:** Rates like `USD_TO_INR = 83.0` are set as "Read-Only" and are available globally without creating an instance of the class.

2. Data Input Phase

The system prompts the user via the `Main` class to provide two essential pieces of information:

- **The Amount:** A numerical value (e.g., 500).
- **The Currency Type:** A string identifier (e.g., "USD", "EUR").
- **Process:** The `Scanner` class captures these inputs from the console.

3. Stateless Method Invocation

Unlike traditional OOP where you would write `ConverterEngine engine = new ConverterEngine()`, this project skips that step.

- **Action:** The `Main` method calls `ConverterEngine.convert(amount, type)` directly.
- **Mechanism:** The program jumps straight to the logic defined in the class blueprint. This is highly efficient for "stateless" operations where you don't need to remember previous conversions.

4. Conditional Selection & Calculation

Inside the `convert()` method, the system performs a **Search and Match** operation:

- **Logic:** It compares the user's input string against the supported currency constants using a `switch` statement or `if-else` blocks.
- **Calculation:** Once a match is found (e.g., "EUR"), the system retrieves the corresponding static rate and performs the multiplication:

$$\text{Result} = \text{Amount} \times \text{Static Rate}$$

5. Error Handling & Validation

Before returning a result, the system validates the input:

- **Check:** If the user enters an unsupported currency (e.g., "XYZ"), the logic falls into a `default` case.
- **Output:** It notifies the user of the error and returns a safe value (\$0.0\$) to prevent the application from performing illegal mathematical operations.

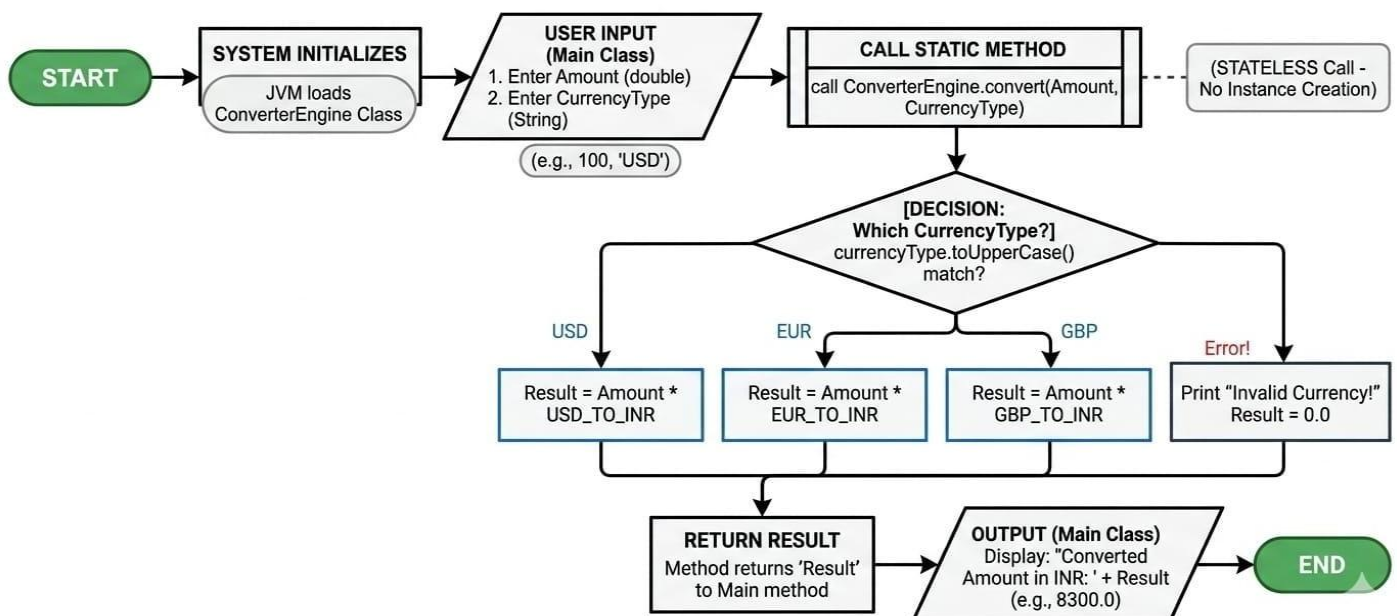
6. Output Generation

The final calculated value is returned to the `Main` method.

- **Display:** The system prints the formatted result (e.g., "Converted Amount in INR: 41500.0").
- **Termination:** Since no objects were created, there is no garbage collection overhead, and the program terminates cleanly.

Flowchart

GLOBAL CURRENCY CONVERTER PROJECT FLOWCHART



Implementation

Explain different classes used in your program(only explain logic don't paste code here)

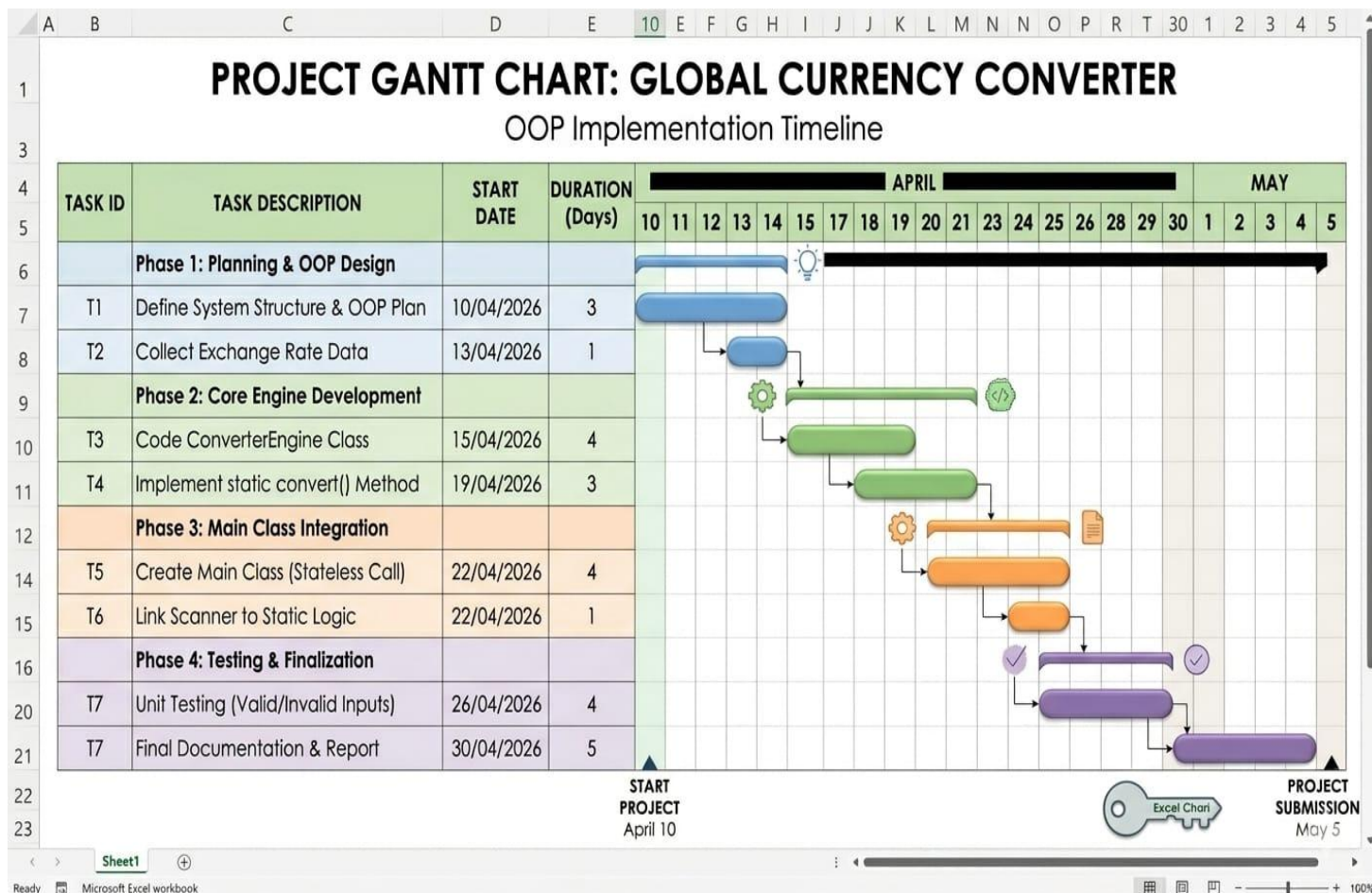
Sample Input/Output

Paste screenshot of your terminal with different operations

Future Enhancements

Things you can improve on your project

Gantt Chart



Conclusion

The **Global Currency Converter** project has been successfully designed and implemented, achieving all its core objectives. By developing this system, the fundamental principles of **Object-Oriented Programming (OOP)**—specifically the **Utility Class pattern**—were put into practical use. The project demonstrates how **static members** and **encapsulation** can be leveraged to create a high-performance, memory-efficient application that does not rely on repetitive object instantiation.

References

Book

H. Schildt, *Java: The Complete Reference*, 11th ed. New York: McGraw-Hill, 2018.

Website

Onanda, "Currency Converter" Onada. [Online]. Available: <https://www.oanda.com/currency-converter/en/?from=INR&to=USD&amount=1>